

GUÍA DE LABORATORIO
(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	CONSTRUCCION DE SOFTWARE				
TÍTULO DE LA PRÁCTICA:	Fundamentos de arquitectura de software: drivers, estilos arquitectónicos y decisiones documentadas con Diagram as Code e IA				
NÚMERO DE PRÁCTICA:	04	AÑO LECTIVO:	2026B	NRO. SEMESTRE:	VI
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	X	MÁXIMO DE ESTUDIANTES	3	
FECHA INICIO:	28/09/2026	FECHA FIN:	03/10/2026	DURACIÓN:	90 min + trabajo semanal
RECURSOS A UTILIZAR: • Computadora con navegador, conexión a internet y cuenta de GitHub. • Visual Studio Code con extensiones: Markdown Preview Mermaid Support y PlantUML. • Editores en línea: https://mermaid.live, https://www.plantuml.com/plantuml y https://play.d2lang.com • Python 3.10+ con las librerías diagrams y matplotlib; Graphviz instalado en el sistema. • Asistente de IA a elección (Claude, ChatGPT, Gemini o GitHub Copilot) y Git.					
DOCENTE(s): <i>Mg. Antonio Arroyo Paz</i>					

OBJETIVOS/TEMAS Y COMPETENCIAS
OBJETIVOS: • Identificar los drivers arquitectónicos de un sistema y especificar atributos de calidad mediante escenarios verificables. • Comparar estilos arquitectónicos y seleccionar uno mediante una matriz de decisión ponderada, apoyándose críticamente en asistentes de IA. • Representar la arquitectura con técnicas de Diagram as Code (Mermaid, PlantUML y Python Diagrams) versionadas en Git. • Documentar las decisiones arquitectónicas mediante Architecture Decision Records (ADR).
TEMAS: • Definición de arquitectura de software (Bass et al.; ISO/IEC/IEEE 42010). • Drivers arquitectónicos y atributos de calidad (ISO/IEC 25010:2023); escenarios de calidad. • Estilos arquitectónicos: capas, cliente-servidor, MVC, monolito modular, microservicios, eventos y hexagonal. • Decisiones, trade-offs y Architecture Decision Records (ADR). • Diagram as Code e inteligencia artificial como apoyo al diseño arquitectónico.

COMPETENCIAS	C.b Genera de forma responsable prototipos, experimentos y modelos, con el fin de analizar e interpretar información para la toma de decisiones fundamentadas y objetivas.
	C.c Diseña responsablemente sistemas, componentes o procesos para satisfacer necesidades dentro de restricciones realistas: económicas, medioambientales, sociales, políticas, éticas, de salud, de seguridad, manufacturación y sostenibilidad.
	C.d Trabaja éticamente, de manera efectiva, en equipos multidisciplinarios, adaptándose a diferentes entornos laborales y nuevas situaciones.
	C.g Comunica con tolerancia, a público de diferentes especialidades, culturas y jerarquías, de forma clara y efectiva, ideas, resultados y productos relacionados con su ejercicio profesional.

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

1.1 ¿Qué es la arquitectura de software?

Bass, Clements y Kazman (2021) definen la arquitectura de software como el **conjunto de estructuras** necesarias para razonar sobre un sistema: sus elementos, las relaciones entre ellos y las propiedades de ambos. La norma **ISO/IEC/IEEE 42010:2022** la entiende como los conceptos o propiedades fundamentales de un sistema en su entorno, expresados en sus elementos, relaciones y principios de diseño y evolución.

Una forma práctica de reconocer una decisión arquitectónica es preguntarse **cuánto costaría cambiarla después**. Elegir el lenguaje de un script es una decisión de diseño; elegir entre un monolito y microservicios, o entre una base de datos relacional y una de documentos, es arquitectura, porque condiciona todo lo que viene después.

1.2 Drivers arquitectónicos

La arquitectura no se elige por moda, sino a partir de los **drivers arquitectónicos**: los requisitos funcionales más relevantes, los **atributos de calidad** y las **restricciones** del proyecto (presupuesto, tamaño y experiencia del equipo, plazos, normativa, tecnología impuesta). De ellos se derivan las decisiones, que siempre implican **compromisos (trade-offs)** y deben quedar registradas (Figura 1).

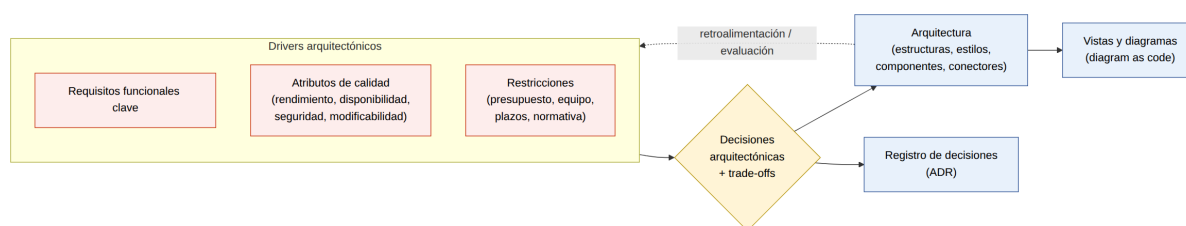


Figura 1. De los drivers a la arquitectura documentada (diagrama elaborado con Mermaid).

1.3 Atributos de calidad y escenarios

La norma **ISO/IEC 25010:2023** organiza la calidad del producto en nueve características: adecuación funcional, eficiencia de desempeño, compatibilidad, capacidad de interacción, fiabilidad, seguridad (security), mantenibilidad, flexibilidad y protección (safety). Como "el sistema debe ser rápido" no es verificable, los atributos de calidad se especifican mediante **escenarios de atributo de calidad** de seis partes (Bass et al., 2021):

Parte	Pregunta que responde	Ejemplo (rendimiento)
Fuente	¿Quién o qué genera el estímulo?	300 compradores de Arequipa
Estímulo	¿Qué ocurre?	Consultan el catálogo de leche disponible
Entorno	¿En qué condiciones?	Hora pico (5:00–7:00 a. m.), operación normal

Artefacto	¿Qué parte del sistema lo recibe?	API del módulo Catálogo
Respuesta	¿Qué debe hacer el sistema?	Devuelve la lista filtrada por distrito
Medida	¿Cómo se verifica?	p95 del tiempo de respuesta ≤ 2 s

1.4 Estilos (patrones) arquitectónicos

Un **estilo arquitectónico** es una solución probada que define tipos de componentes, conectores y restricciones de organización. Ningún estilo es "el mejor": cada uno favorece ciertos atributos de calidad y perjudica otros (Richards y Ford, 2020).

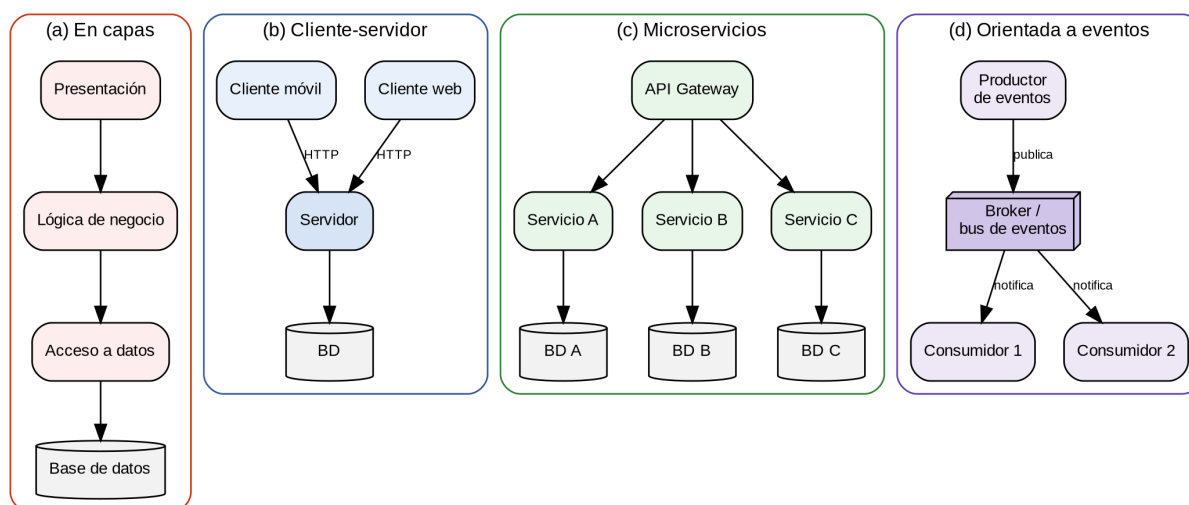


Figura 2. Cuatro estilos arquitectónicos frecuentes (diagrama elaborado con Graphviz/DOT).

Estilo	Idea central	Favorece	Penaliza / riesgos
En capas (n-tier)	Responsabilidades separadas en capas; cada capa usa solo la inferior.	Simplicidad, separación de responsabilidades, costo bajo.	Escalabilidad fina; tiende a "monolito de barro" si no se cuidan las dependencias.
Cliente-servidor	Clientes solicitan servicios a un servidor central.	Centralización de datos y control.	El servidor es cuello de botella y punto único de falla.
MVC	Separa modelo, vista y controlador en la capa de presentación.	Mantenibilidad de la interfaz y pruebas.	Controladores "gordos" si no hay capa de servicios.
Monolito modular	Un solo despliegue dividido en módulos de dominio con interfaces explícitas.	Simplicidad operativa con buena modificabilidad; camino a microservicios.	Requiere disciplina para respetar los límites entre módulos.
Microservicios	Servicios pequeños, desplegables de forma independiente, cada uno con sus datos.	Escalabilidad y despliegue independientes, equipos autónomos.	Complejidad operativa, red, datos distribuidos, costo alto.
Orientada a eventos	Componentes que publican y reaccionan a eventos mediante un broker.	Desacoplamiento, escalabilidad, reacción en tiempo real.	Depuración difícil, consistencia eventual.
Hexagonal (puertos y adaptadores)	El dominio en el centro; la tecnología se conecta mediante puertos y adaptadores.	Testabilidad, independencia tecnológica.	Más abstracciones y código inicial.

1.5 Decisiones arquitectónicas y ADR

Un **Architecture Decision Record (ADR)** es un documento breve, versionado junto al código, que registra una decisión relevante, su contexto y sus consecuencias (Nygard, 2011). Su valor está en explicar **por qué** se decidió algo, para que el equipo futuro no repita discusiones ni revierta decisiones sin entenderlas.

```
docs/architecture/adr/000-plantilla.md
# ADR-NNN: <Título corto de la decisión>

- Estado: Propuesto | Aceptado | Rechazado | Reemplazado por ADR-XXX
- Fecha: AAAA-MM-DD
- Decisores: <integrantes del equipo>

## Contexto
<Qué problema o driver motiva la decisión: requisitos, atributos de calidad y restricciones.>

## Alternativas consideradas
1. <Alternativa A>
2. <Alternativa B>

## Decisión
<Qué se decidió, en voz activa: "Usaremos...">

## Consecuencias
- Positivas: <...>
- Negativas / riesgos: <...>
```

1.6 Diagram as Code

Diagram as Code consiste en describir los diagramas con un lenguaje de texto que una herramienta convierte en imagen. Frente a dibujar con el mouse, ofrece ventajas decisivas: los diagramas se **versionan en Git** junto al código, se revisan en un Pull Request, muestran diferencias (diff) línea por línea, se regeneran automáticamente y son un formato ideal para trabajar con **asistentes de IA**, que generan y corrigen texto con facilidad.

Herramienta	Fortaleza	Render / editor	Uso en esta guía
Mermaid	Sintaxis simple; se renderiza nativamente en GitHub, GitLab y Notion.	mermaid.live, extensión de VS Code, mermaid-cli	Diagramas de estructura (Figuras 1, 3 y 4)
PlantUML	Muy completo para UML; gran ecosistema (incluye C4-PlantUML).	plantuml.com, extensión de VS Code, plantuml.jar	Alternativa descartada (Figura 5)
Python Diagrams	Diagramas de infraestructura con íconos reales (nube, BD, colas).	Python + Graphviz	Vista de despliegue (Figura 6)
Graphviz (DOT)	Motor de layout clásico y muy flexible.	dot, GraphvizOnline	Estilos arquitectónicos (Figura 2)
D2 / Structurizr DSL	Diagramas modernos (D2) y modelos C4 con una sola fuente (Structurizr).	play.d2lang.com, structurizr.com	Exploración opcional (Structurizr se retoma en el Tema 08)

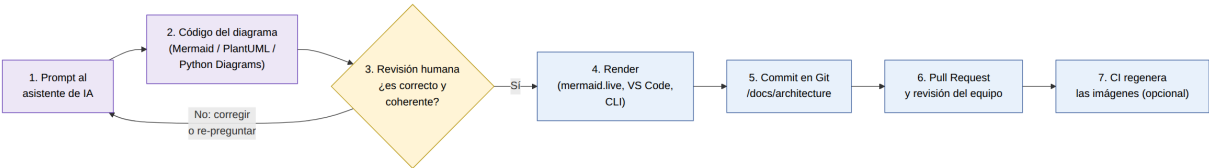


Figura 3. Flujo de trabajo propuesto: IA + Diagram as Code + Git (diagrama elaborado con Mermaid).

1.7 Inteligencia artificial como apoyo al diseño arquitectónico

Los asistentes de IA (Claude, ChatGPT, Gemini, GitHub Copilot) son útiles para **generar alternativas, redactar el código de los diagramas, criticar una propuesta** ("abogado del diablo") y **redactar borradores de ADR**. Sin embargo, tienen riesgos concretos: pueden inventar datos o capacidades de herramientas, suelen sesgarse hacia arquitecturas de moda (microservicios, Kubernetes) aunque no convengan, y no conocen las restricciones reales del equipo si no se las damos.

Regla de oro de esta práctica: la IA propone, el equipo decide y verifica. Toda salida de IA que se use debe revisarse, corregirse si es necesario y registrarse en la bitácora de IA. Nunca se ingresan datos personales o confidenciales en los prompts.

Estructura recomendada de prompt (RCRTF): Rol · Contexto · Restricciones · Tarea · Formato de salida.

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Caso: "AgroConecta Majes". La Cooperativa Agraria de Majes necesita una plataforma web y móvil para que pequeños productores de leche y hortalizas publiquen su producción diaria y los compradores de Arequipa (restaurantes, bodegas, mercados) realicen pedidos y paguen con Yape o Plin. El equipo de SurTech Solutions tiene 5 developers, presupuesto limitado y debe lanzar un MVP en 1 mes. Muchos productores usan celulares de gama baja con conectividad intermitente.

Paso 1. Identificar los drivers arquitectónicos

Tipo de driver	Drivers identificados
Requisitos funcionales clave	Registrar producción diaria; catálogo con filtros; carrito y pedidos; pago con Yape/Plin; notificación por WhatsApp; reportes para la cooperativa.
Atributos de calidad	Rendimiento en hora pico; disponibilidad de pedidos aunque falle un servicio externo; modificabilidad para agregar pasarelas de pago; capacidad de interacción en celulares de gama baja.
Restricciones	5 developers (experiencia en Python/Django), presupuesto de hosting bajo, MVP en 1 mes, integración obligatoria con WhatsApp y Yape/Plin.

Paso 2. Especificar escenarios de atributos de calidad

Atributo	Fuente / Estímulo	Entorno	Artefacto	Respuesta	Medida
Rendimiento	300 compradores consultan el catálogo	Hora pico (5–7 a. m.)	Módulo Catálogo	Lista filtrada por distrito	p95 ≤ 2 s
Disponibilidad	La API de WhatsApp no responde	Operación normal	Módulo Notificaciones	El pedido se registra y la notificación se reintentará	0 pedidos perdidos; reintento ≤ 15 min
Modificabilidad	La cooperativa pide agregar una nueva pasarela de pago	Desarrollo	Módulo Pagos	Se agrega un adaptador sin tocar otros módulos	≤ 3 días-persona
Capacidad de interacción	Un productor registra su producción	Celular de gama baja, señal 3G	PWA	La app funciona sin conexión y sincroniza luego	Registro en ≤ 3 toques

Paso 3. Generar alternativas con IA (y verificarlas)

Se usó un asistente de IA con la estructura RCRTF. El prompt incluye explícitamente las restricciones, para evitar que la IA proponga soluciones sobredimensionadas:

Prompt 1 — Generación de alternativas

Actúa como arquitecto de software senior con experiencia en sistemas para PYMES.
Contexto: plataforma "AgroConecta" para una cooperativa lechera de Majes (Arequipa).
Productores publican producción diaria; compradores hacen pedidos y pagan con Yape/Plin;

notificaciones por WhatsApp; ~300 usuarios concurrentes en hora pico.
 Restricciones: 5 developers con experiencia en Python/Django, presupuesto de hosting bajo (un VPS), MVP en 1 mes, productores con celulares de gama baja y 3G.
 Tarea: propon 3 alternativas de estilo arquitectónico. Para cada una indica fortalezas, debilidades, riesgos y qué atributos de calidad favorece o penaliza.
 Formato: tabla comparativa en Markdown y, al final, tu recomendación justificada.
 No inventes APIs ni capacidades de servicios; si no estás seguro, indícalo.

La IA propuso tres alternativas: **(A) monolito en capas, (B) microservicios con Kubernetes y (C) monolito modular**. Luego se aplicó un segundo prompt de **crítica**, pidiendo a la IA actuar como "abogado del diablo" contra la alternativa que ella misma consideró mejor:

Prompt 2 — Crítica adversarial

Ahora actúa como "abogado del diablo". Critica duramente la alternativa que recomendaste: ¿qué supuestos no se cumplen con nuestras restricciones?, ¿qué podría fallar en producción?, ¿qué costo oculto tiene? Enumera los 5 riesgos más graves y, para cada uno, una táctica arquitectónica de mitigación.

Verificación humana (extracto de la bitácora de IA): la IA recomendó inicialmente microservicios "por escalabilidad". El equipo lo descartó: 300 usuarios concurrentes no justifican 5 despliegues, 3 bases de datos y un broker para 5 developers. También se corrigió un dato inventado: la IA afirmó que Yape ofrece una API pública abierta; se verificó que la integración requiere un proveedor de pagos o convenio.

Paso 4. Comparar alternativas con una matriz de decisión ponderada

Criterio (peso)	Capas	Monolito modular	Microservicios	Justificación
Costo operativo (25 %)	5	5	2	Un solo servidor frente a varios servicios y BD
Tiempo de entrega (25 %)	5	4	2	El MVP debe salir en 1 mes
Modificabilidad (20 %)	2	4	5	Agregar pasarelas y módulos
Escalabilidad (15 %)	2	3	5	Carga moderada y predecible
Simplicidad operativa (15 %)	5	4	1	Equipo sin experiencia en DevOps
Total ponderado	3,95	4,10	2,90	Gana el monolito modular

El gráfico se generó con un script de Python (matplotlib), lo que también permite versionarlo y regenerarlo si cambian los pesos:

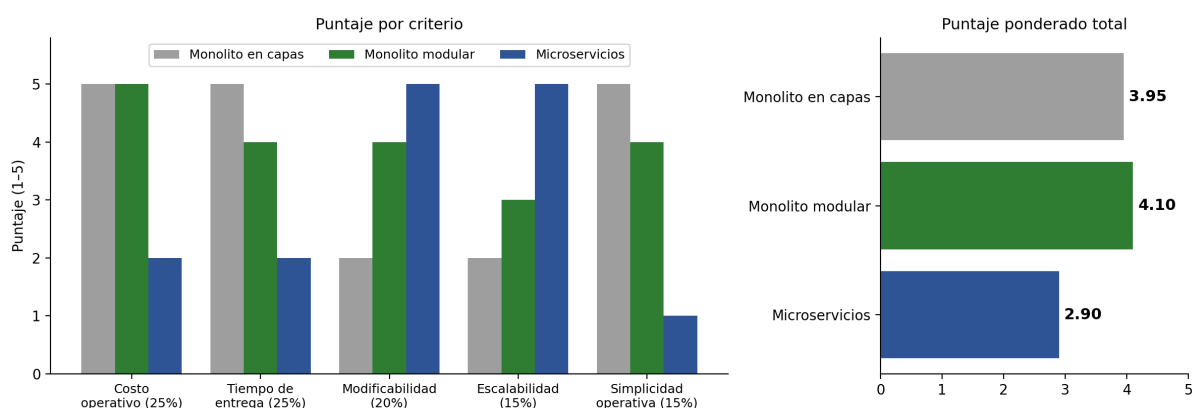


Figura 4. Matriz de decisión ponderada: el monolito modular obtiene el mayor puntaje (4,10).

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 7

Paso 5. Diagrama de la arquitectura elegida con Mermaid

El siguiente código se pega en <https://mermaid.live> o en un archivo `.md` de GitHub, que lo renderiza automáticamente:

`docs/architecture/diagramas/arquitectura.mmd`

```

flowchart TB
    PR["Productor"]
    CO["Comprador"]
    AD["Administrador de la cooperativa"]
    subgraph APP["AgroConecta – Monolito modular (un solo despliegue)"]
        API["Capa de presentación: API REST + PWA"]
        M1["Catálogo y<br/>producción"]
        M2["Pedidos y<br/>carrito"]
        M3["Pagos"]
        M4["Notificaciones"]
        M5["Reportes"]
        INF["Capa de infraestructura: repositorios y adaptadores externos"]
    end
    end
    DB["PostgreSQL<br/>(un esquema por módulo)"]
    YP["Pasarela Yape / Plin"]
    WA["WhatsApp Business API"]
    PR & CO & AD --> API
    API --> M1 & M2 & M3 & M4 & M5
    M1 & M2 & M3 & M4 & M5 --> INF
    INF --> DB
    INF --> YP
    INF --> WA
    classDef mod fill:#E8F5E9,stroke:#2E7D32,color:#000
    classDef ext fill:#F2F2F2,stroke:#7F7F7F,color:#000,stroke-dasharray: 4 3
    classDef usr fill:#FDEDEC,stroke:#C8310E,color:#000
    class M1,M2,M3,M4,M5 mod
    class YP,WA ext
    class PR,CO,AD usr

```

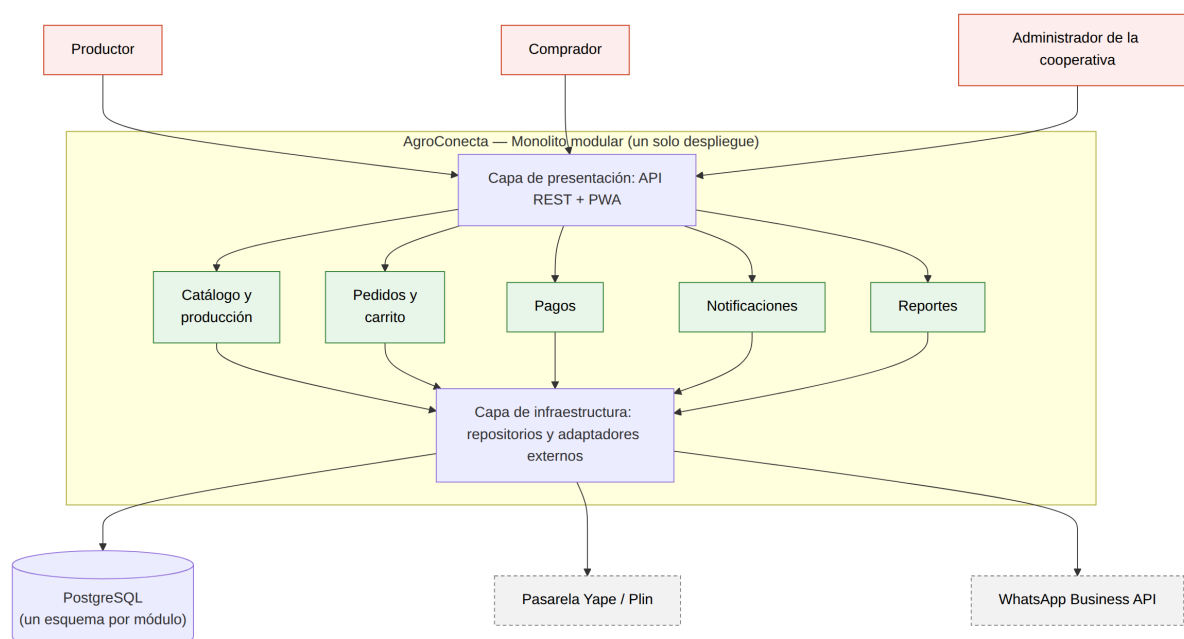


Figura 5. Arquitectura propuesta para AgroConecta: monolito modular (render del código Mermaid anterior).

Paso 6. Documentar la alternativa descartada con PlantUML

Documentar lo que **no** se eligió también es valioso: evita reabrir la discusión y muestra que la decisión fue analizada.

docs/architecture/diagramas/alternativa-microservicios.puml

```
@startuml
skinparam dpi 220
skinparam componentStyle rectangle
skinparam defaultFontName Arial
skinparam backgroundColor white
title Alternativa B (descartada): AgroConecta con microservicios
actor "Productor / Comprador" as U
node "API Gateway" as GW
package "Microservicios" {
    [Catálogo] as C
    [Pedidos] as P
    [Pagos] as PA
    [Notificaciones] as N
    [Reportes] as R
}
queue "Broker de eventos\n(RabbitMQ)" as MQ
database "BD Catálogo" as DB1
database "BD Pedidos" as DB2
database "BD Pagos" as DB3
cloud "Yape / Plin" as Y
cloud "WhatsApp API" as W
U --> GW
GW --> C
GW --> P
GW --> R
C --> DB1
P --> DB2
PA --> DB3
P ..> MQ : PedidoCreado
MQ ..> PA
MQ ..> N
PA --> Y
N --> W
note bottom of MQ
    Mayor escalabilidad independiente,
    pero +5 despliegues, +3 BD, broker,
    monitoreo distribuido: costo alto
    para un equipo de 5 developers.
end note
@enduml
```

Alternativa B (descartada): AgroConecta con microservicios

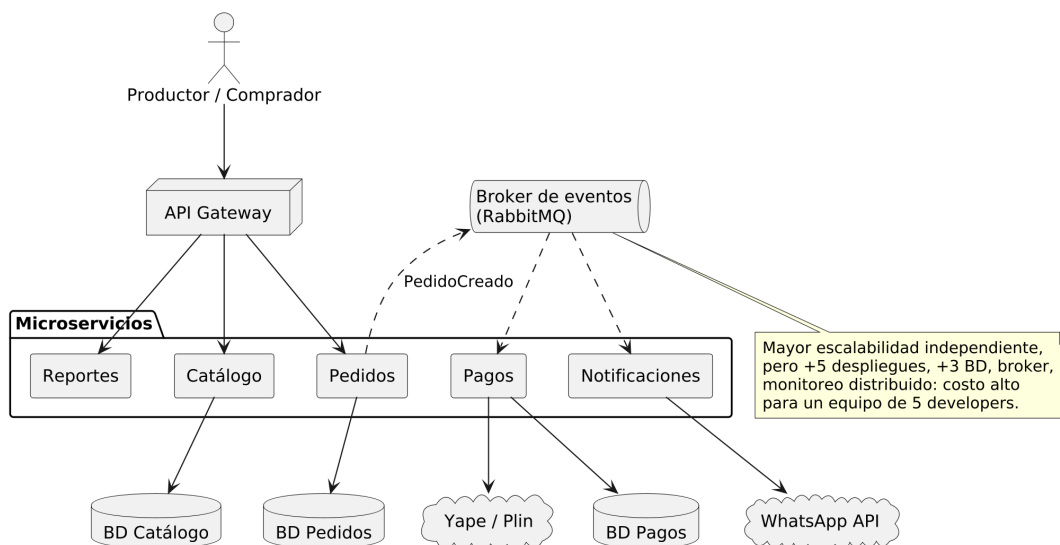


Figura 6. Alternativa B descartada: microservicios (render del código PlantUML anterior).

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 9

Paso 7. Vista de despliegue con Python Diagrams

Con la librería **Diagrams** (mingrammer) la infraestructura se describe en Python con íconos reales. Instalación: `pip install diagrams` (requiere Graphviz instalado en el sistema). Ejecución: `python_despliegue.ipynb`.

`docs/architecture/diagrams/despliegue.ipynb`

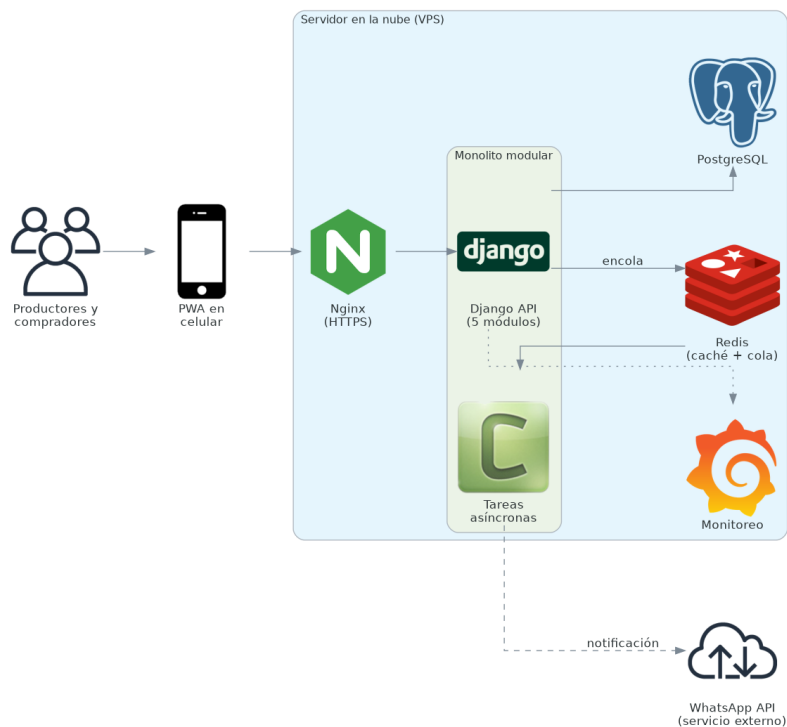
```

pip install diagrams
from diagrams import Diagram, Cluster, Edge
from diagrams.onprem.client import Users
from diagrams.onprem.network import Nginx
from diagrams.programming.framework import Django
from diagrams.onprem.database import PostgreSQL
from diagrams.onprem.inmemory import Redis
from diagrams.onprem.queue import Celery
from diagrams.onprem.network import Internet
from diagrams.generic.device import Mobile
from diagrams.onprem.monitoring import Grafana

graph_attr = {"fontsize": "20", "bgcolor": "white", "pad": "0.3"}
with Diagram("AgroConecta - Vista de despliegue", filename="fig6", show=False,
            direction="LR", graph_attr=graph_attr, outformat="png") as diag:
    usuarios = Users("Productores y\ncompradores")
    movil = Mobile("PWA en\ncelular")
    with Cluster("Servidor en la nube (VPS)":
        proxy = Nginx("Nginx\n(HTTPS)")
        with Cluster("Monolito modular"):
            app = Django("Django API\n(5 módulos)")
            worker = Celery("Tareas\nasíncronas")
            cache = Redis("Redis\n(caché + cola)")
            db = PostgreSQL("PostgreSQL")
            mon = Grafana("Monitoreo")
    usuarios >> movil >> proxy >> app
    app >> db
    app >> Edge(label="en cola") >> cache >> worker
    worker >> Edge(label="notificación", style="dashed") >> Internet("WhatsApp API\n(servicio externo)")
    app >> Edge(style="dotted") >> mon
diag

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 10



AgroConecta - Vista de despliegue

Figura 7. Vista de despliegue de AgroConecta (render del script Python anterior).

Paso 8. Registrar la decisión en un ADR

docs/architecture/adr/001-monolito-modular.md

ADR-001: Adoptar un monolito modular para el MVP de AgroConecta

- Estado: Aceptado
- Fecha: 2026-09-29
- Decisores: equipo SurTech (5 developers) y Product Owner

Contexto

El MVP debe salir en 1 mes con 5 developers y un único VPS. La carga esperada (~300 usuarios concurrentes) es moderada. Se prevé agregar nuevas pasarelas de pago y funcionalidades, por lo que la modificabilidad es importante.

Alternativas consideradas

1. Monolito en capas (3,95): simple, pero los módulos quedarían acoplados.
2. Microservicios (2,90): escalable, pero excede la capacidad operativa del equipo.
3. Monolito modular (4,10): elegido.

Decisión

Usaremos un monolito modular en Django con 5 módulos (Catálogo, Pedidos, Pagos, Notificaciones, Reportes). Los módulos se comunican solo mediante interfaces públicas (servicios de aplicación) y cada uno tiene su propio esquema en PostgreSQL. Las integraciones externas se implementan como adaptadores (puertos y adaptadores).

Consecuencias

- Positivas: un solo despliegue, bajo costo, entrega rápida; los módulos pueden extraerse como microservicios en el futuro si la carga lo exige.
- Negativas / riesgos: el equipo debe respetar los límites entre módulos (se usará import-linter en la CI); una falla grave afecta a todo el sistema.

Paso 9. Estructura del repositorio y contenido de cada archivo

Los códigos **E1 a E8** identifican los **entregables** de los ejercicios propuestos de la Sección III (E1 = Ejercicio 1, E2 = Ejercicio 2, y así sucesivamente). Cada entregable se guarda en un archivo específico del repositorio. El ejercicio resuelto de AgroConecta (Pasos 1 a 8) sirve como **modelo** del contenido esperado en cada archivo.

Estructura del repositorio cs-2026b-lab04-grupoXX	
cs-2026b-lab04-grupoXX/	
└─ README.md	# E8: portada del proyecto
└─ docs/	
└─ architecture/	
└─ drivers.md	# E1: drivers + 3 escenarios de calidad
└─ matriz-decision.md	# E2: 3 alternativas + matriz ponderada
└─ bitacora-ia.md	# E7: prompts usados y verificación
└─ adr/	# E4: decisiones arquitectónicas
└─ 000-plantilla.md	# plantilla (copiar de la sección 1.5)
└─ 001-estilo-arquitectonico.md	
└─ 002-<decision>.md	# p. ej., 002-base-de-datos.md
└─ 003-<decision>.md	# p. ej., 003-autenticacion.md
└─ diagramas/	
└─ arquitectura.mmd	# E3: Mermaid de la alternativa ELEGIDA
└─ alternativa.puml	# E5: PlantUML de una alternativa DESCARTADA
└─ despliegue.py	# E6: vista de despliegue (Python Diagrams)
└─ img/	# PNG/SVG generados de E3, E5 y E6

¿Qué va dentro de cada archivo?

Código	Archivo	Contenido mínimo obligatorio	Modelo
E1	drivers.md	(1) Tabla de requisitos funcionales clave (mínimo 5). (2) Atributos de calidad priorizados (mínimo 4). (3) Tabla de restricciones (plazo de 1 mes, equipo, presupuesto, normativa, tecnología). (4) Tres escenarios de calidad en tabla de seis partes, con medida numérica.	Pasos 1 y 2
E2	matriz-decision.md	(1) Tres alternativas de estilo, descritas en 2–3 líneas. (2) Mínimo 5 criterios con pesos que sumen 100 % y la justificación de cada peso. (3) Puntajes de 1 a 5 y total ponderado. (4) Conclusión con la alternativa elegida y enlace al ADR-001.	Pasos 3 y 4
E3	diagramas/arquitectura.mmd	Código Mermaid de la alternativa elegida: actores, módulos o componentes, almacenamiento de datos y servicios externos. Su imagen se guarda en img/.	Paso 5
E4	adr/001, 002, 003	Tres ADR completos: estado, fecha, decisores, contexto (citando los IDs de drivers.md), alternativas, decisión y consecuencias positivas y negativas.	Paso 8 y sección 1.5
E5	diagramas/alternativa.puml	Código PlantUML de la segunda mejor alternativa de la matriz, con una nota (note) de 3 a 5 líneas que explique por qué se descartó.	Paso 6
E6	diagramas/despliegue.py	Script de Python Diagrams con usuarios, servidor, aplicación, base de datos, servicios externos y monitoreo; su imagen en img/.	Paso 7
E7	bitacora-ia.md	Tabla con mínimo 5 interacciones con IA: herramienta, prompt, propuesta, verificación y decisión (aceptada, corregida o rechazada).	Paso 3
E8	README.md	Caso, integrantes y roles, diagrama Mermaid embebido, enlaces a los ADR y reflexión grupal de 5 a 8 líneas sobre el uso de la IA.	Plantilla abajo

A continuación se presentan las **plantillas** de los archivos Markdown. Cópielas a su repositorio y reemplace el texto entre < >.

docs/architecture/drivers.md — Plantilla (E1)
Drivers arquitectónicos – <Nombre del caso>

> Pegue los prompts completos debajo de la tabla (sección "Anexo: prompts").
 > Nunca incluya datos personales ni información confidencial en un prompt.

README.md — Plantilla (E8)

```
# <Nombre del caso> – Laboratorio 04: Fundamentos de arquitectura de software
Construcción de Software · EPIS-UNSA · 2026-B · Grupo XX

## Integrantes
| Nombre | Rol en el laboratorio (p. ej., redactor de ADR, diagramador, verificador de IA) |
|-----|-----|

## Caso
<Descripción de 4-6 líneas y atributo de calidad crítico>

## Arquitectura elegida
```mermaid
<pegar aquí el contenido de docs/architecture/diagramas/arquitectura.mmd>
```

## Decisiones arquitectónicas
- [ADR-001: Estilo arquitectónico](docs/architecture/adr/001-estilo-arquitectonico.md)
- [ADR-002: ...](docs/architecture/adr/002-....md)
- [ADR-003: ...](docs/architecture/adr/003-....md)

## Reflexión sobre el uso de la IA (5-8 líneas)
<¿En qué ayudó? ¿Qué errores cometió? ¿Qué aprendimos a verificar?>
```

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Organización: grupos de hasta 3 estudiantes. Cada grupo elige **un caso** de la Tabla de casos (no se repite el caso dentro del mismo horario de laboratorio) y crea un repositorio público en GitHub llamado **cs-2026b-1ab04-grupoXX**, donde XX es el número de su grupo. Agregue al docente como colaborador.

Condiciones comunes a todos los casos

- **Plazo:** el MVP debe estar en producción en **1 mes**. Con los asistentes de IA y la generación de código, este plazo es realista, pero obliga a elegir una arquitectura que el equipo pueda construir y operar rápidamente.
- **Equipo:** los integrantes de su grupo (máximo 3 developers), con las tecnologías que ya dominan.
- **Presupuesto:** bajo; se debe justificar cualquier servicio de pago.
- **Regla de oro:** la IA propone, el equipo decide y verifica. Todo uso de la IA se registra en la bitácora (E7).

Casos disponibles

| N.º | Caso | Contexto y actores | Funcionalidades del MVP | Atributo de calidad crítico (dato medible) |
|-----|--------------------|---|--|---|
| 1 | CitaSalud Arequipa | Reserva de citas en los centros de salud de la región. Actores: paciente, personal de admisión, médico. | Buscar disponibilidad por especialidad; reservar y cancelar citas; recordatorio por WhatsApp; lista de pacientes del día para el médico. | Disponibilidad y rendimiento: 5000 pacientes intentan reservar entre las 7:00 y 7:15 a. m.; p95 ≤ 3 s y ninguna cita asignada dos veces. |
| 2 | PicanteríaYa | Delivery de picanterías arequipeñas. Actores: cliente, picantería, repartidor. | Menú del día por picantería; pedido y pago con Yape; asignación de repartidor; seguimiento en el mapa. | Escalabilidad: los pedidos se multiplican por 8 los domingos de 12:00 a 15:00 (hasta 400 pedidos por hora). |

| | | | | |
|----|---------------------|--|---|---|
| 3 | AireMisti IoT | Red de 40 sensores de calidad del aire (PM10/PM2.5) en Arequipa. Actores: sensor, ciudadano, analista municipal. | Ingesta de lecturas cada minuto; mapa en tiempo real; alerta pública cuando PM10 supere 150 µg/m ³ ; descarga del histórico. | Ingesta y fiabilidad: 0 lecturas perdidas ante cortes de red de hasta 2 horas (almacenamiento local en el sensor). |
| 4 | BiblioUNSA | Préstamo y reserva de libros de las bibliotecas de la UNSA. Actores: estudiante, bibliotecario, sistema académico. | Catálogo en línea; reserva de libros; préstamo con carné QR; control de multas; validación de matrícula vigente. | Seguridad e interoperabilidad: acceso con el correo institucional y consulta al sistema académico mediante API, sin acceso directo a su base de datos. |
| 5 | VotoEPIS | Elección digital de delegados estudiantiles de la EPIS. Actores: estudiante votante, comité electoral, auditor. | Padrón electoral; emisión de voto; conteo automático; publicación de resultados y acta. | Seguridad e integridad: un voto por estudiante, voto secreto y resultados auditables (el conteo debe poder verificarse de forma independiente). |
| 6 | RutaSIT Arequipa | Seguimiento en tiempo real de los buses del Sistema Integrado de Transporte. Actores: pasajero, bus (GPS), operador. | Ubicación de los buses en el mapa; tiempo estimado de llegada al paradero; alertas de desvío o congestión. | Rendimiento en tiempo real: 300 buses envían su posición cada 10 s; el tiempo estimado de llegada se actualiza en ≤ 15 s. |
| 7 | San Camilo en Línea | Pedidos a los puestos del Mercado San Camilo, con recojo o delivery. Actores: cliente, comerciante, repartidor. | Catálogo por puesto; pedido a varios puestos; pago con Yape; confirmación por WhatsApp al comerciante. | Capacidad de interacción (usabilidad): un comerciante con poca experiencia digital publica un producto en ≤ 3 toques desde un celular de gama baja. |
| 8 | SismoReporta AQP | Reporte ciudadano de daños tras un sismo, para apoyar a Defensa Civil. Actores: ciudadano, brigadista, coordinador. | Reporte con foto y geolocalización; mapa de daños; priorización de zonas; asignación de brigadas. | Disponibilidad ante picos extremos: 20 000 reportes en la primera hora con la red móvil congestionada; la app guarda el reporte y lo envía al recuperar señal. |
| 9 | ChacraSmart Majes | Riego inteligente con sensores de humedad y válvulas en parcelas de Majes. Actores: agricultor, sensor, válvula, técnico. | Lectura de humedad del suelo; programación del riego; apertura y cierre remoto de válvulas; alertas por falta de agua. | Fiabilidad y protección (safety): si se pierde la conexión, ninguna válvula queda abierta más del tiempo programado (falla segura). |
| 10 | EcoRecicla AQP | Recojo de residuos reciclables con recicladores formalizados en distritos de Arequipa. Actores: vecino, reciclador, municipalidad. | Solicitud de recojo; ruta del día para cada reciclador; puntos canjeables para los vecinos; reporte de toneladas recicladas. | Modificabilidad: incorporar un nuevo distrito o una nueva regla de puntos en ≤ 2 días-persona, sin modificar los demás módulos. |

Caso propio: el grupo puede proponer el sistema, previa aprobación del docente, completando la misma información de la tabla.

E1. Drivers arquitectónicos y escenarios de calidad — drivers.md

Objetivo: identificar qué factores deben guiar la arquitectura de su caso y expresarlos de forma verificable.

1. Copie la plantilla de **drivers.md** (Paso 9) en **docs/architecture/**.
2. Liste **mínimo 5 requisitos funcionales clave** con su actor. Parta de la columna "Funcionalidades del MVP" de su caso y agregue las que falten.
3. Identifique **mínimo 4 atributos de calidad** y ordénelos por prioridad. El primero debe ser el atributo crítico de su caso; explique en una línea por qué cada uno es importante.
4. Registre **mínimo 4 restricciones**. Son obligatorias R-01 ("MVP en producción en 1 mes") y la restricción de equipo (número de integrantes y tecnologías que dominan).

| | | |
|---|---|---|
|  | <p style="text-align: center;">UNIVERSIDAD NACIONAL DE SAN AGUSTIN
 FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
 ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA</p> |  |
| <p style="text-align: center;">Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación</p> | | |
| <p>Aprobación: 2022/03/01</p> | <p>Código: GUIA-PRLD-001</p> | <p>Página: 15</p> |

5. Redacte **3 escenarios de calidad** de atributos distintos (uno debe ser el atributo crítico), con sus seis partes. Use el dato medible de su caso como referencia.

6. Haga commit con el mensaje **E1: drivers y escenarios de calidad**.

Lista de verificación:

- ☐ Ninguna medida usa palabras vagas como "rápido", "seguro" o "fácil": todas son números (tiempo, porcentaje, cantidad).
- ☐ Cada escenario puede probarse; se sabe cómo medir si se cumple.
- ☐ Las restricciones son reales para su equipo, no copiadas del ejemplo.

E2. Alternativas con IA y matriz de decisión — matriz-decision.md

Objetivo: comparar al menos tres estilos arquitectónicos y elegir uno con criterios explícitos, usando la IA como apoyo y no como autoridad.

1. Adapte el **Prompt 1** (Paso 3): reemplace el contexto y las restricciones por los de su **drivers.md**. Incluya explícitamente el plazo de 1 mes y el tamaño de su equipo.
2. Pida **3 alternativas** de estilo arquitectónico. Registre el prompt y un resumen de la respuesta en **bitacora-ia.md** (entrada 1).
3. Aplique el **Prompt 2** (crítica adversarial) a la alternativa que la IA recomendó. Regístrelo en la bitácora (entrada 2).
4. Defina **mínimo 5 criterios** derivados de sus drivers (p. ej., tiempo de entrega, costo, atributo crítico, simplicidad operativa, modificabilidad). Asigne pesos que sumen 100 % y justifique cada peso citando un driver (R-01, QA-01, etc.).
5. Puntúe cada alternativa de 1 a 5 en cada criterio y calcule el total ponderado. Opcionalmente, genere el gráfico con matplotlib, como en la Figura 4.
6. Identifique **al menos una afirmación de la IA** incorrecta, exagerada o que no respete sus restricciones, y explique cómo lo verificaron.
7. Haga commit con el mensaje **E2: matriz de decision**.

Lista de verificación:

- ☐ Los pesos suman exactamente 100 % y cada uno tiene justificación.
- ☐ La alternativa ganadora es coherente con el plazo de 1 mes y con la experiencia del equipo.
- ☐ La decisión final la tomó el equipo; si difiere de la recomendación de la IA, se explica por qué.

E3. Diagrama como código de la alternativa elegida — arquitectura.mmd

Objetivo: representar la arquitectura elegida con Mermaid, de forma versionable.

1. Escriba el diagrama o pídaselo a la IA a partir de su **matriz-decision.md**; en ese caso, **revise línea por línea** lo que generó.
2. Valide el código en <https://mermaid.live> y corrija los errores de sintaxis.
3. El diagrama debe incluir: **mínimo 2 actores**, todos los módulos o componentes principales, el almacenamiento de datos, **mínimo 1 servicio externo** y la dirección de las dependencias. Use **subgraph** para agrupar capas o módulos.
4. Exporte la imagen (PNG o SVG) a **diagramas/img/** y haga commit con el mensaje **E3: diagrama Mermaid**.

Lista de verificación:

- ☐ El diagrama renderiza sin errores en mermaid.live y en GitHub.

| | | |
|---|---|---|
|  | <p style="text-align: center;">UNIVERSIDAD NACIONAL DE SAN AGUSTIN
 FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
 ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA</p> |  |
| <p style="text-align: center;">Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación</p> | | |
| <p>Aprobación: 2022/03/01</p> | <p>Código: GUIA-PRLD-001</p> | <p>Página: 16</p> |

- ☐ Cada módulo del diagrama corresponde al menos a un requisito funcional de drivers.md.

Parte B — Trabajo autónomo (29/09/2026 al 03/10/2026)

E4. Registros de decisiones arquitectónicas — adr/

1. Copie la plantilla `000-plantilla.md` (sección 1.5).
2. Redacte `001-estilo-arquitectonico.md` con la decisión de E2 (**obligatorio**).
3. Redacte **dos ADR adicionales** con decisiones relevantes de su caso. Por ejemplo: base de datos relacional o documental; estrategia de autenticación; comunicación síncrona o con colas; PWA o app nativa; protocolo de ingesta MQTT o HTTP (casos IoT); nube o servidor propio.
4. En cada ADR: el **Contexto** cita los IDs de `drivers.md` (RF-, QA-, R-); se comparan **mínimo 2 alternativas**; las **Consecuencias** incluyen al menos una positiva y una negativa; el estado es "Aceptado".

- ☐ Cada ADR se entiende por sí solo, sin tener que leer otro documento.

E5. Alternativa descartada en PlantUML — alternativa.puml

1. Tome la **segunda mejor alternativa** de su matriz de decisión.
2. Diágramela en PlantUML (diagrama de componentes o de despliegue) y válidela en <https://www.plantuml.com/plantuml>.
3. Agregue una nota (**note**) de 3 a 5 líneas que explique por qué se descartó, citando su puntaje en la matriz.
4. Exporte la imagen a `diagramas/img/`.

E6. Vista de despliegue — despliegue.py

1. Instale las herramientas: `pip install diagrams` y Graphviz en el sistema.
2. Represente dónde se ejecuta cada parte: usuarios o dispositivos (celulares, sensores, buses), proxy o balanceador, aplicación, base de datos, caché o colas si aplica, servicios externos y monitoreo. Use **Cluster** para agrupar servidores y etiquete las conexiones con `Edge(label=...)`.
3. Ejecute `python despliegue.py`, guarde la imagen en `diagramas/img/` y haga commit del script y de la imagen.
4. Si no puede instalar Graphviz, elabore la vista con un diagrama de despliegue de PlantUML e indíquelo en el README.

E7. Bitácora de uso de IA — bitacora-ia.md

1. Complete la tabla de la plantilla con **mínimo 5 interacciones** (incluidas las 2 de E2). Pueden ser de generación, corrección de diagramas, revisión de ADR o crítica adversarial.
2. Al menos **una** interacción debe tener la decisión "Rechazada" o "Corregida", con la evidencia de la verificación (documentación oficial, cálculo o restricción que no se cumplía).
3. Pegue los prompts completos en un anexo al final del archivo.

E8. README y revisión cruzada — README.md

1. Complete el `README.md` con la plantilla del Paso 9: caso, integrantes y roles, diagrama Mermaid embebido (GitHub lo renderiza automáticamente), enlaces a los 3 ADR y reflexión grupal de 5 a 8 líneas sobre el aporte y los límites de la IA.
2. **Revisión cruzada:** cada integrante abre al menos **un Pull Request** con uno de los entregables, y otro integrante lo revisa y aprueba en GitHub.
3. Entregue el enlace del repositorio en el Aula Virtual hasta el sábado 03/10/2026 a las 23:59.

| | | |
|--|--|---|
|  | <p align="center">UNIVERSIDAD NACIONAL DE SAN AGUSTIN
 FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
 ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA</p> |  |
| Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación | | |
| Aprobación: 2022/03/01 | Código: GUIA-PRLD-001 | Página: 17 |

Reto opcional (+2 puntos, sin exceder la nota máxima): configure una GitHub Action que, en cada push, regenere automáticamente las imágenes PNG de los archivos .mmd usando mermaid-cli. Otra opción es usar la IA como "arquitecto adversario" en una sesión documentada de al menos 6 turnos y responder con un ADR que refute o acepte sus críticas.

IV. CUESTIONARIO

1. ¿Por qué se afirma que una decisión arquitectónica es aquella "costosa de cambiar"? Dé un ejemplo de su caso.
2. ¿Cuál es la diferencia entre un requisito funcional y un atributo de calidad? ¿Por qué los atributos de calidad influyen más en la arquitectura?
3. Reescriba el requisito "el sistema debe ser seguro" como un escenario de atributo de calidad de seis partes.
4. Compare el monolito modular y los microservicios en términos de costo, modificabilidad y complejidad operativa. ¿En qué momento convendría migrar de uno a otro?
5. ¿Qué ventajas ofrece Diagram as Code frente a herramientas de dibujo como PowerPoint? Mencione al menos tres.
6. ¿Qué elementos debe contener un ADR y por qué es importante registrar también las alternativas descartadas?
7. Describa un caso de esta práctica en el que la IA haya generado una propuesta incorrecta o sesgada. ¿Cómo lo detectaron?
8. ¿Qué riesgos éticos y de confidencialidad existen al usar asistentes de IA para diseñar la arquitectura de un sistema real?

V. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

Bass, L., Clements, P. y Kazman, R. (2021). Software Architecture in Practice (4.ª ed.). Addison-Wesley.

Richards, M. y Ford, N. (2020). Fundamentals of Software Architecture: An Engineering Approach. O'Reilly Media.

Valente, M. T. (2020). Engenharia de Software Moderna, cap. 7: Arquitetura. Editora Independente.
<https://engsoftmoderna.info/>

ISO/IEC/IEEE 42010:2022. Software, systems and enterprise — Architecture description.

ISO/IEC 25010:2023. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model.

Nygaard, M. (2011). Documenting Architecture Decisions. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>

ADR GitHub Organization. Architectural Decision Records. <https://adr.github.io/>

Mermaid. Documentación oficial. <https://mermaid.js.org/> — Editor en línea: <https://mermaid.live/>

PlantUML. Documentación oficial. <https://plantuml.com/>

Diagrams: Diagram as Code for prototyping cloud system architectures. <https://diagrams.mingrammer.com/>

Keeling, M. (2017). Design It! From Programmer to Software Architect. Pragmatic Bookshelf.

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:

- Observación sistemática del trabajo en laboratorio (checkpoint).

INSTRUMENTOS:

- Lista de cotejo del checkpoint (E1–E3).
- Rúbrica analítica del producto final (ver criterios).
- Bitácora de uso de IA.

| | | |
|--|--|---|
|  | <p align="center">UNIVERSIDAD NACIONAL DE SAN AGUSTIN
 FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
 ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA</p> |  |
| Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación | | |
| Aprobación: 2022/03/01 | Código: GUIA-PRLD-001 | Página: 18 |

- Análisis de productos: repositorio, diagramas como código y ADR.
- Revisión entre pares mediante Pull Requests.

CRITERIOS DE EVALUACIÓN

| Criterio | Logrado | En proceso | No logrado | Pts |
|--|---|---|--|-----------|
| Drivers y escenarios de calidad | Drivers completos y 3 escenarios con 6 partes y medidas verificables | Escenarios incompletos o medidas ambiguas | No identifica drivers ni escenarios | 3 |
| Análisis de alternativas | 3 alternativas, matriz ponderada coherente con restricciones | Matriz incompleta o pesos sin justificar | Decisión sin análisis | 4 |
| Diagramas como código | Mermaid, PlantUML y Python Diagrams correctos, versionados y renderizados | Falta un diagrama o hay inconsistencias | Diagramas dibujados a mano o ausentes | 4 |
| ADR | 3 ADR completos, con alternativas y consecuencias | ADR incompletos o superficiales | Sin ADR | 4 |
| Uso crítico de IA | Bitácora ≥ 5 entradas con verificación y correcciones reales | Bitácora sin evidencia de verificación | No declara el uso de IA | 3 |
| Repositorio y colaboración | README claro, commits de todos los integrantes, PR revisados | Commits de un solo integrante | Repositorio desorganizado o entrega tardía | 2 |
| | | | Total | 20 |